Context Integrity in LLM Agents

Two research communities study the same failure. Neither tests the other's work.
We measured what that costs.

Nyamtulla · Advisor meeting · 7 October 2026

An agent believes everything in its context window

An LLM agent reads one block of text and then acts. We call this block the context window.

The block holds the user's question. It also holds a page a tool returned, a document a search step found, and a note the agent stored last week.

The model cannot tell these apart. Everything in the block looks the same to it.

So any text that reaches the block can change what the agent does. The text does not need to come from the user. It only needs to arrive.

This is not about training data. It is about what enters the context while the agent runs.

Two communities. One failure. No contact.

The security literature

It treats the text as an attack. Someone wrote it to take the agent over.

Indirect prompt injection. Poisoned retrieval. Poisoned memory.

The machine learning literature

It treats the same corruption as an accident. Nobody wrote it to cause harm.

Lost in the middle. Distraction by irrelevant text. Knowledge conflict.

Both build defenses. Each side tests its defenses on its own failures.

A tool returns a page with a hidden order. A tool returns a page that is ten times too long. Both arrive through the same route. Both end in the same window. One has an author and one does not, and that is the only difference.

Our question: does that difference justify two separate sets of defenses?

Security end: Greshake et al., AISec@CCS 2023, arXiv:2302.12173 · ML end: Liu et al., TACL 2023, arXiv:2307.03172; Shi et al., ICML 2023, arXiv:2302.00093 · Knowledge conflict: Xu et al., EMNLP 2024, arXiv:2403.08319

We describe every case with three questions

Channel	How did the text get in? Which route did it travel?
Intent	Did a person put it there on purpose, or not?
Consequence	What went wrong once the text was in the window?

9 channels × 3 intents × 6 consequences = 162 combinations.

A combination with no paper in it is a question the field has never asked. We find that 98 of the 162 are empty.

The framework is the spine of the work. It tells us what is known, and it tells us where to look next.

CHANNEL

Nine routes carry text into the window

tool-output	What a tool gives back: a web page, an email, a file the agent opened.	**75**
direct-input	The prompt itself. The user's text, or the system prompt above it.	**67**
RAG	A document a search step pulled out of a corpus.	**28**
cross-modal	Text carried inside an image, audio or video.	**18**
memory	Something the agent stored earlier and now reads again.	**16**
tool-metadata	The description of a tool, not its output.	**13**
multi-agent	A message from another agent in the same system.	**10**
supply-chain	A package or template the agent inherits before it starts.	**6**
skill	A stored procedure the agent loads and follows.	**4**

The right column counts the named attacks in that channel.

tool-output Greshake et al., AISec@CCS 2023 · direct-input Liu et al., 2023 · RAG Zou et al., USENIX Security 2024 · memory Chen et al., NeurIPS 2024 · cross-modal Wang et al., EMNLP 2025
tool-metadata Wang et al., AAAI 2026 · multi-agent Lee et al., ESORICS 2025 · supply-chain Qu et al., 2026 · skill Jia et al., 2026

CONSEQUENCE

Six ways the agent then fails

goal-hijack	The agent does another person's task. Text in the window gives an order and it obeys.	**136**
reasoning-corruption	Every word is true. The window is too long or too cluttered, and the model stops using it well.	**46**
silent-corruption	The agent answers from false content. Nobody gave it a new task. It believes a wrong thing.	**21**
persistence-backdoor	The bad content is stored. It fires on a later run, maybe for another user.	**16**
data-exfiltration	The agent sends out what it can read, through a tool call, a link or a message.	**14**
resource-abuse	No data moves and no goal changes. The agent burns time, or refuses to answer.	**4**

The second row is the machine learning literature. The other five are the security literature. That is the split, in one column.

goal-hijack Greshake et al., AISec@CCS 2023 · reasoning-corruption Liu et al., TACL 2023; Shi et al., ICML 2023 · silent-corruption Xie et al., ICLR 2023 · persistence-backdoor Chen et al., NeurIPS 2024 · data-exfiltration Qi et al., ICLR 2024 · resource-abuse Xue et al., 2024

A map of every attack and every defense in the field

1,026 papers	read in full and coded, drawn from about 26,000 candidates
237 attacks	distinct named ways to corrupt a context window
534 defenses	distinct named ways the field protects against them
349 pairs	every case where a paper tested one defense against one attack

No prior survey holds both literatures. The closest one reviews 78 papers, 37 attacks and 41 defenses, and stays on the security side.

That map is the instrument. Everything that follows is read off it.

FINDING 1

The two communities do not read each other

11.7 %	of security papers cite at least one machine learning paper
9.1 %	of machine learning papers cite at least one security paper
98.0 %	of all tested pairs stay inside one literature
60.5 %	of the 162 combinations in the framework have never been studied

Nine of ten papers never cross. Seven pairs in the whole field do.

This is not a sample. We resolved every citation edge in the corpus.

So every claim of the form “this defense is robust” holds inside one threat model. Nobody has checked the other one.

FINDING 2

Most defenses have never met most attacks

119 of 237 attacks have no tested defense at all

15 of 534 defenses were tested against both threat models. That is 2.8 %.

A defense paper is not a neutral document. A defense is supposed to defend something.

99 defense papers name an attack that sits in our map, and then never test against it. Those are not gaps in the field's knowledge. They are gaps in its evidence.

At this point the survey had taken us as far as a survey can go. It can show that a test was never run. It cannot say what the test would have found.

Where an untested pair is waiting

A route and a failure, with attacks on one side and defenses on the other. The last column is how many of those pairs anyone has tested.

Route in	What goes wrong	Attacks	Defenses	Pairs tested
direct-input	reasoning-corruption	4	79	**0**
RAG	silent-corruption	3	42	**0**
tool-output	goal-hijack	61	1	**0**
RAG	reasoning-corruption	2	17	**0**
direct-input	goal-hijack	27	1	**0**
direct-input	silent-corruption	1	11	**0**
memory	persistence-backdoor	7	1	**0**

Every row is zero. Not one of these pairs has been tested by anybody.

Take the second row. Three attacks poison a retrieved document so the agent answers wrongly. Forty-two defenses answer the same failure when it happens by accident. 126 possible tests. None run.

Released code is not the same as working code

We only report a number if we can run the authors' own code. So the first question is which code runs at all.

Attacks

4 run

1 runs

4 do not run missing data, a missing prompt file, a checkpoint with no tokenizer

Defenses

Four rebuilt and running and give a usable measurement.

One rebuilt and running, but cannot reproduce its own published benefit, so we report nothing from it

Nine defense codebases attempted. Four produced a measurement.

That is not a complaint about other people's repositories. It sets the size of what this work can honestly cover, and it is a result the field should know.

The map tells us which pairs can even be run

A defense and an attack belong together when they sit in the same place on the map: the same route in, and the same thing going wrong.

If a defense was built for a failure that happens by accident, and an attack causes the same failure on purpose through the same route, then the defense has a claim to answer. Nobody has made it answer.

The cell we worked in

Route in	a document returned by a search step
What goes wrong	the agent answers from false content and nothing signals it
On the attack side	3 named attacks, all deliberate
On the defense side	42 named defenses, all built for the accidental version
Tested against each other	none of them, by anyone

Every defense we ran lives in that cell. So does every attack. We did not pick them because they looked promising. The map picked them.

The first measurements anyone has taken in that cell

4 attacks	rebuilt from released code and papers
5 defenses	counting two versions of one of them, all from released code
18 pairs	defense against attack, on three models, none of them reported anywhere
104 conditions	each pair run at several attack strengths, not one

Two rules decide whether we are allowed to claim anything.

A defense must first show its own benefit on its own task. If it cannot, we report nothing from it.

We fix in advance the smallest change the sample can detect. Below that we say nothing, in either direction.

Scoring is the authors' own code, unchanged.

THE RESULT

Does the defense stop the attack?

It stops the attack	the attack succeeds less often with the defense switched on	**4**
It does nothing	no readable difference either way	**7**
It makes the attack worse	the attack succeeds MORE often with the defense switched on	**7**

out of 18 pairs

A defense helped in four cases out of eighteen. In seven it made the attack work better than no defense at all.

In 4 of those seven the attack does almost nothing to a model with no defense at all. The defense is what makes it work.

That is the finding. A defense built for one kind of failure is not neutral against the other kind. It can be the reason the attack succeeds.

What is finished and what is still running

Done

1 cell worked through: the retrieved document, answering from false context

18 pairs measured, none of them previously tested

3 models including a controlled repeat of the same pair on a second model

Running now

5 models testing why one model breaks and another does not

Left to do

One more attack in this cell: a third poisoning method with released code.

An aggregation defense from a different research group, to check the rule is not one laboratory's habit.

Then the next cell on the map. The largest untested pool is a different route in, with four attacks and seventy-nine defenses facing each other and nothing between them.

99 defense papers name an attack on our map and never test against it. Those are the next candidates after that.

The cell we finished took four attacks and five defenses. The map holds 10 more cells with both sides present and nothing tested between them.